Coding Sample: Escaping Inequality in India

Selected Excerpts

Rishav Roy

What follows are excerpts from the 138-page code appendix to my independent paper and writing sample, "Escaping Inequality in India: The Role of English-Medium Instruction."

The next 47 pages include the following excerpts, each a part of the code chunks below it:

14-16. Scalable **import, cleaning, and merging** of multi-file census and geospatial data.

- Chunk 4: Import key data files

31-32. **QA and identifier-disambiguation checks** for district-level crosswalk.

- Chunk 6: Match districts: Construct district tracking dfs

50-52. **Testing for systematic missingness** using the Benjamini–Hochberg procedure and parallelized logistic regressions.

- Chunk 8: Participation model: Are NAs randomly distributed

55-57. **Survey-weighted probit estimation**, average marginal effects derivation, and setup for sample selection correction under a complex survey design.

- Chunk 9: Participation model: Estimation and IMR
- Chunk 10: Participation model: Derive AME

80-88. Reusable **record-linkage functions** using cascading fuzzy matches and unmatched/false-match diagnostics to harmonize district identifiers across years and data sources.

- Chunk 16: Match districts: Test joining methods
- Chunk 17: Match districts: Define joining functions

105-107. Reusable **IV specification and estimation pipeline** with clustered inference, plus contiguity-based spatial weights and setup for spatially lagged models.

- Chunk 24: Geospatial: Neighbor list construction
- Chunk 25: 2SLS: Controls and 2SLS formula
- Chunk 26: 2SLS: Baseline 2SLS models

111-112. **Robustness checks and diagnostics** for multicollinearity, spatial autocorrelation.

- Chunk 28: Multicollinearity check
- Chunk 29: Geospatial: Spatial autocorrelation tests

117-138. **Automated generation of publication-quality figures and tables** (including both summary statistics and regression outputs).

- Chunks 31-40: [Output for paper]

Scalable import, cleaning, and merging of Census/geospatial data

Source: `R/io/read_inputs.R`

```
#' read nss 2007 education
#'
read_nss_2007_education <- function(paths) {
  read_manifest_group(paths, "nss_2007_education")
}

#' read nss 2007 consumption
#'
read_nss_2007_consumption <- function(paths) {
  read_manifest_group(paths, "nss_2007_consumption")
}

#' read nss 2017 education
#'
read_nss_2017_education <- function(paths) {
  read_manifest_group(paths, "nss_2017_education")
}

#' read census 2001 mother tongue
#'
read_census_2001_mother_tongue <- function(paths) {
  read_manifest_group(paths, "census_2001_mother_tongue")
}

#' read district boundaries 2020
#'
read_district_boundaries_2020 <- function(paths) {
  rows <- require_manifest_files(paths, "district_boundaries_2020")
  shp <- rows[tolower(rows$file_type) == "shp", , drop = FALSE]
  if (!nrow(shp)) stop("The district-boundary manifest includes shapefile sidecars but no
   ↪  .shp row.", call. = FALSE)
  need_pkg("sf", "district boundaries")
  sf::st_read(shp$absolute_path[[1]], quiet = TRUE)
}

#' read district change sources
#'
read_district_change_sources <- function(paths) {
  read_manifest_group(paths, "district_changes")
}

#' list ilo figure paths
#'
list_ilo_figure_paths <- function(paths) {
  rows <- require_manifest_files(paths, "ilo_figures")
  stats::setNames(rows$absolute_path, rows$file_id)
}

#' read one manifest row
#'
#' @return Reader output for raw data files, or a validated path for sidecars/assets.
read_by_manifest_row <- function(row) {
```

```
  path <- row$absolute_path[[1]]
  file_type <- tolower(row$file_type[[1]])
  switch(
    file_type,
    sav = read_sav_short(path),
    csv = read_csv_short(path),
    xls = read_excel_short(path),
    xlsx = read_excel_short(path),
    ods = read_ods_short(path),
    shp = {
      need_pkg("sf", "shapefiles")
      sf::st_read(path, quiet = TRUE)
    },
    shx = path,
    dbf = path,
    prj = path,
    png = path,
    stop("No reader implemented for ", file_type, call. = FALSE)
  )
}

#' read all manifest rows for a source
#'
#' @return Named list of reader outputs keyed by manifest file_id.
read_manifest_group <- function(paths, source_id) {
  rows <- require_manifest_files(paths, source_id)
  stats::setNames(lapply(seq_len(nrow(rows)), function(i) read_by_manifest_row(rows[i,
  ↪ ])), rows$file_id)
}
```

QA and identifier-disambiguation checks

Source: `R/districts/build_district_tracker.R`

```
#' build district tracker
#'
build_district_tracker <- function(raw_district_changes) {
  combine_district_tracker_sources(raw_district_changes)
}

#' parse alluvial district changes
#'
parse_alluvial_district_changes <- function(x) {
  x
}

#' parse india district tracker
#'
parse_india_district_tracker <- function(x) {
  x
}

#' parse carveouts renamings
#'
parse_carveouts_renamings <- function(x) {
  x
```

```
}

#' parse new districts created
#'
parse_new_districts_created <- function(x) {
  x
}

#' parse name changes
#'
parse_name_changes <- function(x) {
  x
}

#' parse district splits
#'
parse_district_splits <- function(x) {
  x
}

#' combine district tracker sources
#'
combine_district_tracker_sources <- function(raw_district_changes) {
  out <- safe_bind_rows(lapply(names(raw_district_changes), function(name) {
    x <- safe_df(raw_district_changes[[name]])
    x$source_file_id <- name
    x
  }))
  if (nrow(out)) out$.row_in_source <- ave(seq_len(nrow(out)), out$source_file_id, FUN =
   ↪  seq_along)
  out
}

#' standardize tracker years
#'
standardize_tracker_years <- function(tracker) {
  tracker
}

#' standardize tracker names
#'
standardize_tracker_names <- function(tracker) {
  tracker
}
```

Parallelized missingness diagnostics with BH correction

Source: `R/selection/diagnose_missingness.R`

```
#' diagnose missingness
#'
diagnose_missingness <- function(selection_data, cfg) {
  data.frame(
    missing_var = names(selection_data)[vapply(selection_data, function(x) any(is.na(x)),
     ↪  logical(1))],
    stringsAsFactors = FALSE
```

```
  )
}

#' check missing logit parallel
#'
check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  rhs <- paste(covars, collapse = " + ")
  fit_one <- function(m) { f <- stats::as.formula(paste0("is.na(", m, ") ~ ", rhs)); fit
 ↪  <- stats::glm(f, data = df, family = stats::binomial); broom::tidy(fit) |>
 ↪  dplyr::mutate(missing_var = m, pseudoR2 = 1 - fit$deviance / fit$null.deviance, nobs
 ↪  = stats::nobs(fit)) }
  out <- dplyr::bind_rows(lapply(miss_vars, fit_one))
  out |> dplyr::group_by(missing_var) |> dplyr::mutate(p_adj = {adj <- rep(NA_real_,
 ↪  dplyr::n()); idx <- term != "(Intercept)"; adj[idx] <- p.adjust(p.value[idx], method
 ↪  = method_p); adj}) |> dplyr::ungroup()
}

#' summarize missingness by variable
#'
summarize_missingness_by_variable <- function(df) {
  data.frame(
    missing_var = names(df),
    n_missing = vapply(df, function(x) sum(is.na(x)), integer(1)),
    stringsAsFactors = FALSE
  )
}

#' run missingness logits
#'
run_missingness_logits <- function(df, miss_vars, covars) {
  check_missing_logit_parallel(df, miss_vars, covars)
}

#' adjust missingness pvalues bh
#'
adjust_missingness_pvalues_bh <- function(df) {
  df
}

#' save missingness diagnostics
#'
save_missingness_diagnostics <- function(diagnostics) {
  diagnostics
}
```

Survey-weighted probit and IMR setup

Source: `R/selection/estimate_selection_probit.R`

```
legacy_probit_variables <- function(selection_data) {
  controls <- c("AGE", "age", "SEX", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "SECTOR")
  exclusion_all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ")
  exclusion_district <- c(
    "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
    "dmean_num_RECD_SCHOLARSHIP_STIPEND", "dmean_num_RECD_TXT_BOOKS",
    "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_RECD",
```

```
    "dmean_num_ENROLLMENT_COST"
  )
  intersect(c(controls, exclusion_all_kids, exclusion_district), names(selection_data))
}

#' estimate selection probit
#'
estimate_selection_probit <- function(selection_data, cfg) {
  if (!"enrolled" %in% names(selection_data) || all(is.na(selection_data$enrolled))) {
    return(list(status = "out_of_active_pipeline", reason = "No enrolled variable."))
  }
  covars <- legacy_probit_variables(selection_data)
  if (!length(covars)) {
    return(list(status = "out_of_active_pipeline", reason = "No probit covariates."))
  }
  f_probit <- stats::reformulate(covars, response = "enrolled")
  if (identical(cfg$mode, "final") && requireNamespace("survey", quietly = TRUE)) {
    design <- build_survey_design_selection(selection_data)
    if (!is.null(design)) {
      out <- fit_selection_probit(design, f_probit)
      attr(out, "legacy_probit_formula") <- f_probit
      return(out)
    }
  }
  out <- stats::glm(f_probit, data = selection_data, family = stats::binomial(link =
↪   "probit"))
  attr(out, "legacy_probit_formula") <- f_probit
  out
}

#' build survey design selection
#'
build_survey_design_selection <- function(selection_df) {
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(c("STATE", "STRATUM", "SUB_STRATUM_NO"), names(selection_df))
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) return(NULL)
  options(survey.lonely.psu = "average")
  selection_df$.survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,
    nest = TRUE
  )
}

#' fit selection probit
#'
fit_selection_probit <- function(selection_design, f_probit) {
  survey::svyglm(f_probit, design = selection_design, family = stats::quasibinomial(link
↪   = "probit"))
}
```

```r
#' compute inverse mills ratio
#'
compute_inverse_mills_ratio <- function(model, selection_df, f_probit = NULL) {
  if (is.null(f_probit)) f_probit <- attr(model, "legacy_probit_formula") %||%
   ↪  stats::formula(model)
  needed <- all.vars(f_probit)
  needed <- intersect(needed, names(selection_df))
  comp_cases <- stats::complete.cases(selection_df[, needed, drop = FALSE])
  lp <- rep(NA_real_, nrow(selection_df))
  lp[comp_cases] <- stats::predict(model, newdata = selection_df[comp_cases, , drop =
 ↪  FALSE], type = "link")
  phi_lp <- stats::dnorm(lp)
  Phi_lp <- pmin(pmax(stats::pnorm(lp), .Machine$double.eps), 1 - .Machine$double.eps)
  enrolled <- as.character(selection_df$enrolled)
  selection_df$IMR <- ifelse(enrolled == "Yes", phi_lp / Phi_lp, ifelse(enrolled == "No",
 ↪  phi_lp / (1 - Phi_lp), NA_real_))
  selection_df
}

#' tidy selection model
#'
tidy_selection_model <- function(model) {
  broom::tidy(model)
}
```

Average marginal effects via automatic differentiation

Source: R/selection/compute_average_marginal_effects.R

```r
# Use automatic differentiation for SE delta-method gradients, as in the legacy Rmd.

#' compute average marginal effects
#'
compute_average_marginal_effects <- function(selection_model, selection_data, cfg =
 ↪  list()) {
  if (!inherits(selection_model, "glm")) {
    return(ame_out_of_pipeline(
      selection_model$status %||% "out_of_active_pipeline",
      selection_model$reason %||% "Selection model not estimated in smoke mode"
    ))
  }

  # `selection_model` is often a survey::svyglm object loaded from the targets
  # store. Survey lonely-PSU handling is an R option, not a durable model
  # attribute, so it may not be set in the downstream AME target even though it
  # was set when the model was estimated. Set it explicitly around all AME
  # computations to make reruns deterministic and warning/error-free.
  old_lonely_psu <- getOption("survey.lonely.psu")
  old_domain_lonely <- getOption("survey.adjust.domain.lonely")
  options(survey.lonely.psu = "average", survey.adjust.domain.lonely = TRUE)
  on.exit({
    if (is.null(old_lonely_psu)) {
      options(survey.lonely.psu = NULL)
    } else {
      options(survey.lonely.psu = old_lonely_psu)
    }
```

```
    if (is.null(old_domain_lonely)) {
      options(survey.adjust.domain.lonely = NULL)
    } else {
      options(survey.adjust.domain.lonely = old_domain_lonely)
    }
  }, add = TRUE)

#' run expression while muffling survey lonely-PSU warnings
#'
#' survey emits lonely-PSU warnings even when survey.lonely.psu is set to
#' average and the adjustment is intentional. Muffle only those handled
#' warnings inside the AME target so strict final builds remain warning-clean.
muffle_survey_lonely_psu_warnings <- function(expr) {
  withCallingHandlers(
    expr,
    warning = function(w) {
      msg <- conditionMessage(w)
      lonely_psu_warning <- grepl(
        "has only one PSU at stage",
        msg,
        fixed = TRUE
      )
      if (lonely_psu_warning) invokeRestart("muffleWarning")
    }
  )
}


  if (!isTRUE(cfg$run_full_ame)) {
    return(format_ame_results(data.frame(
      term = names(stats::coef(selection_model)),
      estimate = unname(stats::coef(selection_model)),
      std.error = NA_real_,
      statistic = NA_real_,
      p.value = NA_real_,
      s.value = NA_real_,
      conf.low = NA_real_,
      conf.high = NA_real_,
      method = "coefficient_fallback",
      status = "estimated",
      reason = "Draft config uses coefficient fallback instead of full AME computation"
    )))
  }

  if (!requireNamespace("marginaleffects", quietly = TRUE)) {
    return(ame_out_of_pipeline(
      "out_of_active_pipeline",
      "Package marginaleffects is required for full AME computation"
    ))
  }

  out <- muffle_survey_lonely_psu_warnings(
    tryCatch(
      compute_ames_autodiff(selection_model, selection_data),
      error = function(e) {
```

```
        fallback <- compute_ames_probit_analytic(selection_model, selection_data)
        fallback$method <- "delta_method_analytic_probit"
        fallback$reason <- NA_character_
        fallback
      }
    )
  )
  format_ame_results(out)
}

ame_out_of_pipeline <- function(status, reason) {
  data.frame(
    term = NA_character_,
    estimate = NA_real_,
    std.error = NA_real_,
    statistic = NA_real_,
    p.value = NA_real_,
    s.value = NA_real_,
    conf.low = NA_real_,
    conf.high = NA_real_,
    method = "not_run",
    status = status,
    reason = reason
  )
}

#' extract model-frame data and AME weights
#'
#' marginaleffects warns, correctly, when weighted/survey models are summarized
#' without an explicit `wts` argument. Build a newdata frame from the exact
#' model estimation sample and attach a synthetic weight column whenever the
#' fitted model exposes usable weights.
ame_model_weight <- function(model_data, model_weights = NULL) {
  weight_cols <- intersect(c("weight", "WEIGHT", "Multiplier", "multiplier",
 ↪  "(weights)"), names(model_data))
  if (length(weight_cols)) {
    return(suppressWarnings(as.numeric(model_data[[weight_cols[[1]]]])))
  }

  if (!is.null(model_weights) && length(model_weights) == nrow(model_data)) {
    return(suppressWarnings(as.numeric(model_weights)))
  }

  NULL
}

#' build marginaleffects newdata and weights
#'
#' @return List with `data`, `wts`, and `has_explicit_weights`.
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))
  model_weights <- tryCatch(stats::weights(model), error = function(e) NULL)
  weight <- ame_model_weight(model_data, model_weights)
```

```r
  if (!is.null(weight) && length(weight) == nrow(model_data) && any(is.finite(weight) &
   ↪  weight != 1)) {
    model_data$.ame_weight <- weight
    return(list(data = model_data, wts = ".ame_weight", has_explicit_weights = TRUE))
  }

  list(data = model_data, wts = FALSE, has_explicit_weights = FALSE)
}

#' run marginaleffects while muffling only a redundant explicit-weight warning
#'
run_avg_slopes <- function(model, model_data, wts) {
  withCallingHandlers(
    marginaleffects::avg_slopes(
      model,
      newdata = model_data,
      wts = wts,
      vcov = TRUE,
      type = "response"
    ),
    warning = function(w) {
      msg <- conditionMessage(w)
      explicit_weight_warning <- grepl(
        "normally good practice to specify weights using the `wts` argument",
        msg,
        fixed = TRUE
      )
      if (explicit_weight_warning && !identical(wts, FALSE)) {
        invokeRestart("muffleWarning")
      }
    }
  )
}

#' compute ames autodiff
#'
compute_ames_autodiff <- function(model, newdata) {
  # Use the exact estimation sample retained by glm/svyglm. Passing the full
  # pre-model selection_data can have extra rows omitted during model fitting,
  # which makes marginaleffects recycle weights/covariates silently.
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(model, amed$data, amed$wts)
}

#' compute ames fast draft
#'
compute_ames_fast_draft <- function(model, newdata, n = 200) {
  amed <- ame_model_data_and_weights(model)
  run_avg_slopes(
    model,
    dplyr::slice_sample(amed$data, n = min(n, nrow(amed$data))),
    amed$wts
  )
}
```

```r
#' compute analytic probit AMEs
#'
#' @return Data frame of observed-data average marginal effects.
compute_ames_probit_analytic <- function(model, newdata) {
  coefs <- stats::coef(model)
  coef_table <- as.data.frame(summary(model)$coefficients)
  terms <- setdiff(names(coefs), "(Intercept)")
  if (!length(terms)) return(ame_out_of_pipeline("out_of_active_pipeline", "Selection
   ↪  model has no non-intercept terms."))

  newdata <- stats::model.frame(model)
  weights <- if ("weight" %in% names(newdata)) num(newdata$weight) else rep(1,
 ↪  nrow(newdata))
  eta <- stats::predict(model, type = "link")
  mean_phi <- wmean(stats::dnorm(eta), weights)
  factor_levels <- model$xlevels %||% list()

  rows <- lapply(terms, function(term) {
    factor_var <- names(factor_levels)[vapply(names(factor_levels), function(v)
 ↪  startsWith(term, v), logical(1))]
    estimate <- NA_real_
    if (length(factor_var)) {
      var <- factor_var[[which.max(nchar(factor_var))]]
      level <- sub(paste0("^", var), "", term)
      if (level %in% factor_levels[[var]]) {
        hi <- newdata
        lo <- newdata
        hi[[var]] <- factor(level, levels = factor_levels[[var]])
        lo[[var]] <- factor(factor_levels[[var]][[1]], levels = factor_levels[[var]])
        estimate <- wmean(stats::predict(model, newdata = hi, type = "response") -
 ↪  stats::predict(model, newdata = lo, type = "response"), weights)
      }
    } else {
      estimate <- unname(coefs[[term]]) * mean_phi
    }
    p_col <- intersect(c("Pr(>|z|)", "Pr(>|t|)"), names(coef_table))
    p <- if (term %in% rownames(coef_table) && length(p_col)) coef_table[term,
 ↪  p_col[[1]]] else NA_real_
    se_col <- intersect(c("Std. Error", "std.error"), names(coef_table))
    se_beta <- if (term %in% rownames(coef_table) && length(se_col)) coef_table[term,
 ↪  se_col[[1]]] else NA_real_
    se <- if (is.finite(se_beta)) abs(se_beta * mean_phi) else NA_real_
    z <- if (is.finite(se) && se > 0) estimate / se else NA_real_
    p_out <- if (is.finite(z)) 2 * stats::pnorm(abs(z), lower.tail = FALSE) else p
    data.frame(
      term = term,
      estimate = estimate,
      std.error = se,
      statistic = z,
      p.value = p_out,
      s.value = if (is.finite(p_out) && p_out > 0) -log2(p_out) else NA_real_,
      conf.low = if (is.finite(se)) estimate - 1.96 * se else NA_real_,
      conf.high = if (is.finite(se)) estimate + 1.96 * se else NA_real_,
      method = "delta_method_analytic_probit",
      status = "estimated",
```

```
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  })
  do.call(rbind, rows)
}


is_marginaleffects_object <- function(x) {
  inherits(x, c("marginaleffects", "comparisons", "avg_comparisons", "slopes",
   ↪  "avg_slopes", "predictions"))
}

copy_first_ame_column <- function(out, target, candidates) {
  if (!target %in% names(out)) out[[target]] <- NA
  for (candidate in candidates) {
    if (!candidate %in% names(out)) next
    missing_target <- is.na(out[[target]])
    if (any(missing_target)) out[[target]][missing_target] <-
     ↪  out[[candidate]][missing_target]
  }
  out
}

normalize_ame_columns <- function(out) {
  # marginaleffects has used both dotted and snake_case names across versions
  # and model classes. Normalize here before selecting the public/audit schema
  # so uncertainty columns are not silently dropped downstream.
  aliases <- list(
    std.error = c("std_error", "std.err", "Std. Error"),
    statistic = c("z", "z.value", "z_value", "t", "t.value", "t_value"),
    p.value = c("p_value", "p", "Pr(>|z|)", "Pr(>|t|)"),
    s.value = c("s_value"),
    conf.low = c("conf_low", "conf.low", "2.5 %"),
    conf.high = c("conf_high", "conf.high", "97.5 %")
  )
  for (target in names(aliases)) {
    out <- copy_first_ame_column(out, target, aliases[[target]])
  }
  out
}

legacy_ame_lookup <- function() {
  data.frame(
    term = c(
      "AGE", "SEX", "HH_SIZE",
      "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION", "RELIGION",
      "SOCIAL_GROUP", "SOCIAL_GROUP", "SOCIAL_GROUP",
      "SECTOR",
      "DIST_FROM_NEAREST_PRIMARY_CLASS", "DIST_FROM_NEAREST_PRIMARY_CLASS",
       ↪  "DIST_FROM_NEAREST_PRIMARY_CLASS", "DIST_FROM_NEAREST_PRIMARY_CLASS",
      "father_educ", "father_educ", "father_educ", "father_educ", "father_educ",
       ↪  "father_educ", "father_educ",
      "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED",
       ↪  "dmean_num_RECD_SCHOLARSHIP_STIPEND",
```

```
      "dmean_num_RECD_TXT_BOOKS", "dmean_num_RECD_STATIONERY",
       ↪  "dmean_num_MID_DAY_MEAL_ETC_RECD", "dmean_num_ENROLLMENT_COST"
    ),
    contrast = c(
      "dY/dX", "Female - Male", "dY/dX",
      "Muslim - Hindu", "Christian - Hindu", "Sikh - Hindu", "Jain - Hindu", "Buddhist -
       ↪  Hindu", "Zoroastrian - Hindu", "Other - Hindu",
      "Scheduled Tribe - Other", "Scheduled Caste - Other", "Other Backward Class -
       ↪  Other",
      "Urban - Rural",
      "1km <= d <2kms - d<1km", "2kms<= d <3kms - d<1km", "3kms <= d <5kms - d<1km",
       ↪  "d>=5kms - d<1km",
      "Literate, no school - Illiterate", "Literate, school < primary - Illiterate",
       ↪  "Primary - Illiterate", "Upper primary - Illiterate", "Secondary - Illiterate",
       ↪  "Higher secondary - Illiterate", "Postsecondary+ - Illiterate",
      "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX", "dY/dX"
    ),
    Term = c(
      "Age (years)", "Female (ref: Male)", "Household size",
      "Religion: Muslim (ref: Hindu)", "Religion: Christian", "Religion: Sikh",
       ↪  "Religion: Jain", "Religion: Buddhist", "Religion: Zoroastrian", "Religion:
       ↪  Other",
      "Social group: Scheduled Tribe (ref: Other)", "Social group: Scheduled Caste",
       ↪  "Social group: Other Backward Class",
      "Urban (ref: Rural)",
      "Distance 1-2km (ref: <1km)", "Distance 2-3km", "Distance 3-5km", "Distance > 5km",
      "Father's educ.: Literate, no school (ref: Illiterate)", "Father's educ.: Literate,
       ↪  school < primary", "Father's educ.: Primary", "Father's educ.: Upper primary",
       ↪  "Father's educ.: Secondary", "Father's educ.: Higher secondary", "Father's
       ↪  educ.: Postsecondary+",
      "Educ. free available (ref: No)", "Tuition waiver received", "Scholarship/Stipend
       ↪  received", "Textbook(s) received", "Stationery received", "Mid-day meal, etc.
       ↪  received", "Enrollment cost (Rs.)"
    ),
    stringsAsFactors = FALSE
  )
}

legacy_ame_label_order <- function() legacy_ame_lookup()$Term

legacy_ame_modelsummary_label <- function(x) {
  x <- as.character(x)
  x <- gsub("\u00d7", ":", x, fixed = TRUE)
  x <- gsub("\\s+:\\s+", ": ", x, perl = TRUE)
  x <- gsub("\\(ref:\\s*", "(ref: ", x, perl = TRUE)
  x
}

infer_ame_contrast <- function(out) {
  if ("contrast" %in% names(out)) return(as.character(out$contrast))
  contrast <- rep("dY/dX", nrow(out))
  if (!"term" %in% names(out)) return(contrast)
  if ("comparison" %in% names(out)) contrast <- as.character(out$comparison)
  contrast[is.na(contrast) | !nzchar(contrast)] <- "dY/dX"
  contrast
```

```
}

ame_factor_base_terms <- function() {
  unique(legacy_ame_lookup()$term[legacy_ame_lookup()$contrast != "dY/dX"])
}

normalize_encoded_factor_ame_terms <- function(out) {
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  if (!"term" %in% names(out)) return(out)
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(out)

  lookup <- legacy_ame_lookup()
  factor_terms <- ame_factor_base_terms()
  factor_terms <- factor_terms[order(nchar(factor_terms), decreasing = TRUE)]

  for (i in seq_len(nrow(out))) {
    current_term <- as.character(out$term[[i]])
    current_contrast <- as.character(out$contrast[[i]])
    if (!nzchar(current_term) || (!is.na(current_contrast) && current_contrast !=
     ↪  "dY/dX")) next

    direct <- lookup$term == current_term & lookup$contrast == current_contrast
    if (any(direct)) next

    for (base_term in factor_terms) {
      if (!startsWith(current_term, base_term)) next
      level <- trimws(sub(paste0("^", base_term), "", current_term))
      if (!nzchar(level)) next
      candidates <- lookup[lookup$term == base_term, , drop = FALSE]
      contrast_hit <- candidates$contrast[startsWith(candidates$contrast, paste0(level, "
↪  - "))]
      if (length(contrast_hit)) {
        out$term[[i]] <- base_term
        out$contrast[[i]] <- contrast_hit[[1]]
        break
      }
    }
  }
  out
}

attach_legacy_ame_labels <- function(out) {
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(out)
  out <- normalize_encoded_factor_ame_terms(out)
  lookup <- legacy_ame_lookup()
  labeled <- merge(out, lookup, by = c("term", "contrast"), all.x = TRUE, sort = FALSE)
  labeled$Term[is.na(labeled$Term)] <- labeled$term[is.na(labeled$Term)]
  labeled$Term <- factor(labeled$Term, levels = unique(c(legacy_ame_label_order(),
↪  labeled$Term)))
  labeled <- labeled[order(labeled$Term), , drop = FALSE]
  labeled$Term <- as.character(labeled$Term)
  labeled
}
```

```r
legacy_modelsummary_marginaleffects_object <- function(out, native_ame) {
  if (!is_marginaleffects_object(native_ame)) return(NULL)
  out <- as.data.frame(out, stringsAsFactors = FALSE)
  required <- c("Term", "estimate", "std.error", "statistic", "p.value", "conf.low",
 ↪  "conf.high")
  if (!all(required %in% names(out))) return(native_ame)

  ms <- out[, required, drop = FALSE]
  ms$term <- legacy_ame_modelsummary_label(ms$Term)
  ms$contrast <- ""
  ms$Term <- NULL
  # Preserve the marginaleffects class and non-structural attributes so
  # modelsummary can use its native marginaleffects/tidy pathway.  We still
  # hand modelsummary the labeled, ordered rows that are validated for the
  # public table, because passing the raw object caused modelsummary to reorder
  # contrasts and display labels against the wrong estimates.
  native_attributes <- attributes(native_ame)
  structural_attributes <- c("names", "row.names", "class")
  for (nm in setdiff(names(native_attributes), structural_attributes)) {
    attr(ms, nm) <- native_attributes[[nm]]
  }
  class(ms) <- unique(c(class(native_ame), class(ms)))
  ms
}

#' format ame results
#'
format_ame_results <- function(ame_results) {
  native_ame <- ame_results
  out <- tibble::as_tibble(ame_results)
  if (!"term" %in% names(out) && "variable" %in% names(out)) out$term <- out$variable
  if (!"contrast" %in% names(out)) out$contrast <- infer_ame_contrast(as.data.frame(out))
  out <- normalize_ame_columns(out)
  if (!"method" %in% names(out)) out$method <- "autodiff"
  if (!"status" %in% names(out)) out$status <- "estimated"
  if (!"reason" %in% names(out)) out$reason <- NA_character_
  legacy_terms <- unique(legacy_ame_lookup()$term)
  use_legacy_schema <- any(out$term %in% legacy_terms) ||
    any(grepl("^dmean_num_", out$term %||% character(), perl = TRUE)) ||
    any((out$contrast %||% "dY/dX") != "dY/dX", na.rm = TRUE)

  if (isTRUE(use_legacy_schema)) {
    out <- attach_legacy_ame_labels(out)
    required <- c(
      "Term", "term", "contrast", "estimate", "std.error", "statistic", "p.value",
       ↪  "s.value",
      "conf.low", "conf.high", "method", "status", "reason"
    )
  } else {
    required <- c(
      "term", "estimate", "std.error", "statistic", "p.value",
      "s.value", "conf.low", "conf.high", "method", "status", "reason"
    )
```

```
  }

  for (nm in setdiff(required, names(out))) out[[nm]] <- NA
  if ("p.value" %in% names(out) && "s.value" %in% names(out)) {
    missing_s <- is.na(out$s.value) & is.finite(out$p.value) & out$p.value > 0
    out$s.value[missing_s] <- -log2(out$p.value[missing_s])
  }
  out <- out[, required, drop = FALSE]
  if (is_marginaleffects_object(native_ame)) {
    attr(out, "legacy_marginaleffects") <-
     ↪  legacy_modelsummary_marginaleffects_object(out, native_ame)
  }
  out
}

#' save ame results
#'
save_ame_results <- function(ame_results, path =
 ↪  "outputs/tables/diagnostics/ame_results.csv") {
  readr::write_csv(tibble::as_tibble(ame_results), path); path
}
```

Cascading fuzzy district record linkage

Source: `R/districts/fuzzy_join_districts.R`

```
# The functions below preserve the cascading fuzzy-match architecture from the legacy
 ↪  Rmd.
# sample-end marker appears near the end of this file after the functions are migrated.

#' evaluate distances
#'
evaluate_distances <- function(pairs, methods, thresholds, col1 = "str1", col2 = "str2")
 ↪  {
  if (length(methods) != length(thresholds)) stop("\"methods\" and \"thresholds\" must
   ↪  have the same length.")
  safe_bind_rows(lapply(seq_along(methods), function(i) {
    distance <- utils::adist(canon(pairs[[col1]]), canon(pairs[[col2]]), ignore.case =
 ↪  TRUE)[, 1]
    data.frame(
      str1 = pairs[[col1]],
      str2 = pairs[[col2]],
      method = methods[i],
      distance = distance,
      threshold = thresholds[i],
      match = distance <= thresholds[i],
      stringsAsFactors = FALSE
    )
  }))
}

#' fuzzy join sequence
#'
fuzzy_join_sequence <- function(df1, df2, dist1, state1, dist2, state2, methods,
 ↪  thresholds, mode = "full") {
```

```r
  df1_id <- df1 |> dplyr::mutate(.id1 = dplyr::row_number())
  df2_id <- df2 |> dplyr::mutate(.id2 = dplyr::row_number())
  df1_curr <- df1_id; df2_curr <- df2_id; matched_all <- tibble::tibble()
  for (i in seq_along(methods)) {
    full_j <- fuzzyjoin::stringdist_join(df1_curr, df2_curr, by =
 ↪  stats::setNames(c(dist2, state2), c(dist1, state1)), mode = mode, method =
 ↪  methods[i], max_dist = thresholds[i], distance_col = "dist")
    matched_i <- full_j |> dplyr::filter(!is.na(.id1), !is.na(.id2)) |>
 ↪  dplyr::group_by(.id1) |> dplyr::slice_min(dist, n = 1, with_ties = FALSE) |>
 ↪  dplyr::ungroup() |> dplyr::group_by(.id2) |> dplyr::slice_min(dist, n = 1, with_ties
 ↪  = FALSE) |> dplyr::ungroup()
    matched_all <- dplyr::bind_rows(matched_all, matched_i)
    df1_curr <- df1_curr |> dplyr::filter(!(.id1 %in% matched_i$.id1)); df2_curr <-
 ↪  df2_curr |> dplyr::filter(!(.id2 %in% matched_i$.id2))
  }
  list(joined = matched_all, unmatched_df1 = df1_curr |> dplyr::select(-.id1),
   ↪  unmatched_df2 = df2_curr |> dplyr::select(-.id2))
}

#' unmatched rows
#'
#' @return A data frame of unmatched rows when the join map carries that attribute.
unmatched_rows <- function(join_map, ...) {
  attr(join_map, "unmatched_rows") %||% join_map[0, , drop = FALSE]
}

#' merge dfs into tracker
#'
#' Legacy-compatible cascading district matcher. `df_names` is a named list (or a
#' data frame) whose elements carry source-specific district/state names.  For
#' each source we try every requested tracker year in order, keep one-to-one
#' canonical name matches, and then fuzzy-match remaining rows within state.
#'
#' @return A list with `joined_df`, `unmatched_df`, and `flagged_df`, matching
#' the object contract used by the legacy Rmd.
merge_dfs_into_tracker <- function(df_names, tracker = NULL, years_of_interest =
 ↪  c("2001", "2007", "2008", "2017", "2018", "2020"), flag = TRUE, ...) {
  if (is.null(tracker)) return(list(joined_df = safe_df(df_names), unmatched_df =
   ↪  data.frame(), flagged_df = data.frame()))
  tracker <- safe_df(tracker)
  if (!nrow(tracker)) return(list(joined_df = data.frame(), unmatched_df =
   ↪  safe_df(df_names), flagged_df = data.frame()))
  if (!".tracker_row" %in% names(tracker)) tracker$.tracker_row <- seq_len(nrow(tracker))
  xs <- if (inherits(df_names, "data.frame")) list(source = df_names) else
 ↪  as_input_list(df_names)

  joined_all <- list()
  unmatched_all <- list()
  flagged_all <- list()
  for (source_name in names(xs)) {
    source <- safe_df(xs[[source_name]])
    if (!nrow(source)) next
    source$.source_row <- seq_len(nrow(source))
    source$.source_name <- source_name
    source <- add_source_name_keys(source)
```

```
    if (!all(c(".source_state_key", ".source_district_key") %in% names(source))) {
      unmatched_all[[source_name]] <- source
      next
    }

    source$.matched <- FALSE
    for (yr in years_of_interest) {
      sfx <- substr(as.character(yr), 3, 4)
      state_col <- paste0("state_", sfx)
      district_col <- paste0("district_", sfx)
      if (!all(c(state_col, district_col) %in% names(tracker))) next
      candidate <- tracker[c(".tracker_row", state_col, district_col)]
      candidate$.tracker_state_key <- canon(candidate[[state_col]])
      candidate$.tracker_district_key <- canon(candidate[[district_col]])
      exact <- merge(
        source[!source$.matched, , drop = FALSE],
        candidate,
        by.x = c(".source_state_key", ".source_district_key"),
        by.y = c(".tracker_state_key", ".tracker_district_key"),
        all.x = FALSE,
        all.y = FALSE
      )
      if (!nrow(exact)) next
      exact <- exact[!duplicated(exact$.source_row) & !duplicated(exact$.tracker_row), ,
↪   drop = FALSE]
      if (!nrow(exact)) next
      exact$match_year <- as.character(yr)
      exact$match_status <- "exact_name"
      joined_all[[paste(source_name, yr, sep = "_")]] <- exact
      source$.matched[source$.source_row %in% exact$.source_row] <- TRUE
    }

    if (any(!source$.matched)) {
      fuzzy <- fuzzy_match_remaining_to_tracker(source[!source$.matched, , drop = FALSE],
↪   tracker, years_of_interest)
      if (nrow(fuzzy)) {
        joined_all[[paste0(source_name, "_fuzzy")]] <- fuzzy
        source$.matched[source$.source_row %in% fuzzy$.source_row] <- TRUE
        if (flag) flagged_all[[source_name]] <- fuzzy[fuzzy$possible_false_positive %in%
↪     TRUE, , drop = FALSE]
      }
    }
    unmatched_all[[source_name]] <- source[!source$.matched, , drop = FALSE]
  }
  list(
    joined_df = safe_bind_rows(joined_all),
    unmatched_df = safe_bind_rows(unmatched_all),
    flagged_df = safe_bind_rows(flagged_all)
  )
}

add_source_name_keys <- function(source) {
  state <- first_col(source, c("state", "state_01", "state_07", "state_08", "state_17",
↪   "state_18", "state_20", "state_0708", "state_1718"))
```

```
  district <- first_col(source, c("district", "district_01", "district_07",
 ↪  "district_08", "district_17", "district_18", "district_20", "district_0708",
 ↪  "district_1718", "district_name"))
  if (!is.null(state)) source$.source_state_key <- canon(source[[state]])
  if (!is.null(district)) source$.source_district_key <- canon(source[[district]])
  source
}

fuzzy_match_remaining_to_tracker <- function(source, tracker, years_of_interest) {
  out <- list()
  for (yr in years_of_interest) {
    sfx <- substr(as.character(yr), 3, 4)
    state_col <- paste0("state_", sfx)
    district_col <- paste0("district_", sfx)
    if (!all(c(state_col, district_col) %in% names(tracker))) next
    candidate <- tracker[c(".tracker_row", state_col, district_col)]
    candidate$.tracker_state_key <- canon(candidate[[state_col]])
    candidate$.tracker_district_key <- canon(candidate[[district_col]])
    for (i in seq_len(nrow(source))) {
      pool <- candidate[candidate$.tracker_state_key == source$.source_state_key[[i]], ,
 ↪  drop = FALSE]
      if (!nrow(pool)) next
      dist <- utils::adist(source$.source_district_key[[i]], pool$.tracker_district_key,
 ↪  ignore.case = TRUE)[1, ]
      best <- which.min(dist)
      if (!length(best) || !is.finite(dist[[best]]) || dist[[best]] > 2) next
      row <- cbind(source[i, , drop = FALSE], pool[best, , drop = FALSE])
      row$match_year <- as.character(yr)
      row$match_status <- "fuzzy_name"
      row$dist <- dist[[best]]
      row$possible_false_positive <- dist[[best]] > 0
      out[[length(out) + 1L]] <- row
    }
  }
  x <- safe_bind_rows(out)
  if (nrow(x)) x <- x[!duplicated(x$.source_row) & !duplicated(x$.tracker_row), , drop =
   ↪  FALSE]
  x
}

#' join year to tracker
#'
join_year_to_tracker <- function(source, tracker, years_of_interest, ...) {
  merge_dfs_into_tracker(source, tracker = tracker, years_of_interest =
   ↪  years_of_interest, ...)
}

#' join all district sources
#'
join_all_district_sources <- function(df_names, tracker, years_of_interest, ...) {
  merge_dfs_into_tracker(df_names, tracker = tracker, years_of_interest =
   ↪  years_of_interest, ...)
}

#' flag many to many matches
```

```r
#'
flag_many_to_many_matches <- function(join_map) {
  join_map$many_to_many <- duplicated(join_map$.source_row) |
 ↪  duplicated(join_map$.tracker_row)
  join_map
}

#' score candidate matches
#'
score_candidate_matches <- function(candidates) {
  if (!"dist" %in% names(candidates)) candidates$dist <- 0
  candidates$score <- -num(candidates$dist)
  candidates
}

#' fuzzy join districts
#'
fuzzy_join_districts <- function(district_tracker, district_keys_2001,
 ↪  district_keys_2007, district_keys_2017, district_keys_2020, cfg) {
  tracker <- safe_df(district_tracker)
  if (nrow(tracker) && all(c("state_01", "district_01", "state_07", "district_07",
   ↪  "state_17", "district_17", "state_20", "district_20") %in% names(tracker))) {
    tracker$.tracker_row <- seq_len(nrow(tracker))
    tracker$source <- "legacy_tracker"
    tracker$match_status <- "legacy_tracker_row"
    tracker$possible_false_positive <- FALSE
    tracker$many_to_many <- FALSE
    attr(tracker, "unmatched_rows") <- tracker[0, , drop = FALSE]
    attr(tracker, "possible_false_positives") <- tracker[0, , drop = FALSE]
    attr(tracker, "many_to_many_cases") <- tracker[0, , drop = FALSE]
    return(tracker)
  }

  out <- safe_bind_rows(Map(function(keys, source) {
    if (!nrow(keys)) return(data.frame())
    keys$source <- source
    keys$match_status <- "source_key_unmatched"
    keys$possible_false_positive <- FALSE
    keys$many_to_many <- FALSE
    keys
  }, list(district_keys_2001, district_keys_2007, district_keys_2017,
 ↪  district_keys_2020), c("2001", "2007", "2017", "2020")))
  attr(out, "unmatched_rows") <- out
  attr(out, "possible_false_positives") <- out[out$possible_false_positive, , drop =
   ↪  FALSE]
  attr(out, "many_to_many_cases") <- out[out$many_to_many, , drop = FALSE]
  out
}
```

Contiguity-based spatial weights

Source: `R/diagnostics/diagnose_spatial_weights.R`

```r
#' build spatial weights
#'
```

```r
#' @return A spatial weights object, or an explicit inactive status when geometry is
 ↪  unavailable.
build_spatial_weights <- function(district_panel, cfg) {
  if (!inherits(district_panel, "sf")) {
    return(list(status = "out_of_active_pipeline", reason = "Requires sf geometry."))
  }
  geom <- sf::st_geometry(district_panel)
  coverage <- mean(!sf::st_is_empty(geom))
  if (!is.finite(coverage) || coverage < 0.75) {
    return(list(
      status = "out_of_active_pipeline",
      reason = paste0(
        "Requires validated non-empty geometry for at least 75% of district-panel rows;
         ↪  current coverage is ",
        round(100 * coverage, 1), "%.")
    ))
  }
  spatial_warnings <- character()
  capture_expected_spatial_warning <- function(expr) {
    withCallingHandlers(
      expr,
      warning = function(w) {
        msg <- conditionMessage(w)
        expected <- grepl("no neighbours|sub-graphs", msg, ignore.case = TRUE)
        if (expected) {
          spatial_warnings <<- unique(c(spatial_warnings, msg))
          invokeRestart("muffleWarning")
        }
      }
    )
  }

  nb <- capture_expected_spatial_warning(
    spdep::poly2nb(district_panel[!sf::st_is_empty(geom), , drop = FALSE], queen = FALSE)
  )
  out <- capture_expected_spatial_warning(
    spdep::nb2listw(nb, style = "W", zero.policy = TRUE)
  )
  attr(out, "spatial_warnings") <- spatial_warnings
  out
}

#' diagnose spatial weights
#'
diagnose_spatial_weights <- function(district_panel, spatial_weights, cfg) {
  data.frame(
    diagnostic = "spatial_weights",
    status = spatial_weights$status %||% "constructed",
    warnings = paste(attr(spatial_weights, "spatial_warnings") %||% character(), collapse
     ↪  = "; "),
    stringsAsFactors = FALSE
  )
}

#' compare rook queen contiguity
```

```r
#'
compare_rook_queen_contiguity <- function(district_panel) {
  tibble::tibble()
}

#' summarize islands
#'
summarize_islands <- function(spatial_weights) {
  tibble::tibble()
}

#' summarize neighbor counts
#'
summarize_neighbor_counts <- function(spatial_weights) {
  tibble::tibble()
}

#' save spatial weight diagnostics
#'
save_spatial_weight_diagnostics <- function(diagnostics) {
  diagnostics
}
```

Reusable IV specification and estimation

Source: `R/iv/build_iv_formulas.R`

```r
#' make iv formula
#'
make_iv_formula <- function(dep, endog, instruments, controls = NULL, fixed_effects =
 ↪  NULL) {
  stats::as.formula(paste(
    dep,
    "~",
    paste(c(endog, controls, fixed_effects), collapse = " + "),
    "|",
    paste(c(instruments, controls, fixed_effects), collapse = " + ")
  ))
}

legacy_iv_controls <- function() {
  c(
    "consumption_0708", "gini_cons_0708",
    "pct_urban", "avg_hh_size", "dependency_ratio",
    "pct_fem_head",
    "pct_hindu", "pct_muslim",
    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary", "pct_head_secondary_plus"
  )
}

#' build iv formulas
#'
build_iv_formulas <- function(cfg) {
  controls <- legacy_iv_controls()
```

```
  list(
    consumption = make_iv_formula(
      "consumption_pct_change",
      "EMIE",
      "wavg_ling_degrees",
      controls
    ),
    gini = make_iv_formula(
      "gini_change",
      "EMIE",
      "wavg_ling_degrees",
      controls
    )
  )
}

#' build baseline 2sls formula
#'
build_baseline_2sls_formula <- function(...) make_iv_formula(...)

#' build fd 2sls formula
#'
build_fd_2sls_formula <- function(...) {
  list(status = "out_of_active_pipeline", reason = "First-difference 2SLS formula is
   ↪  deferred until the FD redesign is specified.")
}

#' build state fe 2sls formula
#'
build_state_fe_2sls_formula <- function(...) {
  list(status = "out_of_active_pipeline", reason = "State fixed-effect 2SLS formula is
   ↪  deferred until the state-FE redesign is specified.")
}
```

Multicollinearity and spatial autocorrelation diagnostics

Source: `R/diagnostics/diagnose_multicollinearity.R`

```
#' diagnose multicollinearity
#'
#' @return A data frame with design-matrix condition diagnostics.
diagnose_multicollinearity <- function(district_panel, iv_models, cfg) {
  model <- if (is.list(iv_models) && length(iv_models)) iv_models[[1]] else iv_models
  out <- tryCatch({
    X <- stats::model.matrix(model)
    data.frame(
      test = "kappa",
      n = nrow(X),
      rank = qr(X)$rank,
      columns = ncol(X),
      kappa = kappa(X, exact = TRUE),
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  }, error = function(e) {
```

```
    data.frame(
      test = "kappa",
      n = NA_integer_,
      rank = NA_integer_,
      columns = NA_integer_,
      kappa = NA_real_,
      status = "out_of_active_pipeline",
      reason = conditionMessage(e),
      stringsAsFactors = FALSE
    )
  })
  out
}

#' compute design matrix rank
#'
compute_design_matrix_rank <- function(formula, data) {
  X <- stats::model.matrix(formula, data = data); tibble::tibble(rank = qr(X)$rank, ncol
 ↪  = ncol(X))
}

#' compute kappa
#'
compute_kappa <- function(formula, data) {
  X <- stats::model.matrix(formula, data = data); kappa(X, exact = TRUE)
}

#' compute vif if applicable
#'
compute_vif_if_applicable <- function(model) {
  if (requireNamespace("car", quietly = TRUE)) car::vif(model) else NULL
}

#' save multicollinearity diagnostics
#'
save_multicollinearity_diagnostics <- function(diagnostics) {
  diagnostics
}
```

Automated figure generation

Source: `R/output/make_figures.R`

```
figure_spec <- function(name, file, title, subtitle = NULL, kind = "status", variable =
 ↪  NULL, sources = NULL, inputs = NULL) {
  out <- list(
    name = name,
    file = file,
    title = title,
    subtitle = subtitle,
    kind = kind,
    variable = variable,
    sources = sources,
    inputs = inputs
  )
}
```

```r
has_sf_geometry <- function(x) {
  inherits(x, "sf") && !is.null(attr(x, "sf_column")) && attr(x, "sf_column") %in%
    ↪ names(x)
}

sf_geometry_coverage <- function(x) {
  if (!has_sf_geometry(x) || !nrow(x)) return(0)
  mean(!sf::st_is_empty(sf::st_geometry(x)))
}

require_final_figure_inputs <- function(district_panel, cfg, required_variables) {
  if (!identical(cfg$mode, "final")) return(invisible(TRUE))
  missing_vars <- setdiff(required_variables, names(as.data.frame(district_panel)))
  if (length(missing_vars)) {
    stop(
      "Final figure generation requires mapped variables. Missing variables: ",
      paste(missing_vars, collapse = ", "),
      call. = FALSE
    )
  }
  if (!has_sf_geometry(district_panel)) {
    stop("Final map generation requires an sf district_panel with validated geometry.",
      ↪ call. = FALSE)
  }
  coverage <- sf_geometry_coverage(district_panel)
  if (!is.finite(coverage) || coverage < 0.75) {
    stop(
      "Final map generation requires a validated geometry join covering at least 75% of
        ↪ district-panel rows; current coverage is ",
      round(100 * coverage, 1),
      "%.",
      call. = FALSE
    )
  }
  invisible(TRUE)
}

#' make figures
#'
#' @return A named list of figure specifications consumed by save_figures().
make_figures <- function(district_panel, raw_ilo_figures, cfg, boundaries_2020 = NULL) {
  required_variables <- c(
    "emie_2007",
    "consumption_growth_pct",
    "pucca_share_2007",
    "head_secondary_plus_2007",
    "region",
    "wavg_ling_degrees"
  )

  out <- list(
    fig_ilo_trends = figure_spec(
      "fig_ilo_trends",
      "fig_ilo_trends.png",
```

```
    "ILO labor market indicators",
    "Composed from the archived ILO figure assets.",
    kind = "ilo_collage",
    sources = raw_ilo_figures
  ),
  district_carveouts_shifts = figure_spec(
    "district_carveouts_shifts",
    "district_carveouts_shifts.png",
    "District carve-outs and shifts",
    kind = "district_carveouts_shifts"
  )
)

missing_vars <- setdiff(required_variables, names(as.data.frame(district_panel)))
geometry_ok <- has_sf_geometry(district_panel) &&
  is.finite(sf_geometry_coverage(district_panel)) &&
  sf_geometry_coverage(district_panel) >= 0.75
maps_available <- !length(missing_vars) && geometry_ok

map_specs <- list(
  map_emi_exposure = figure_spec("map_emi_exposure", "map_emi_exposure.png", "EMI
   Exposure", kind = if (maps_available) "map" else "status", variable =
   "emie_2007"),
  map_consumption_growth = figure_spec("map_consumption_growth",
   "map_consumption_growth.png", "% Change in Consumption", kind = if
   (maps_available) "map" else "status", variable = "consumption_growth_pct"),
  map_pucca = figure_spec("map_pucca", "map_pucca.png", "% Pucca Homes", kind = if
   (maps_available) "map" else "status", variable = "pucca_share_2007"),
  map_education = figure_spec("map_education", "map_education.png", "% HH Head w/
   Sec.+", kind = if (maps_available) "map" else "status", variable =
   "head_secondary_plus_2007"),
  map_region = figure_spec("map_region", "map_region.png", "Region", kind = if
   (maps_available) "map" else "status", variable = "region"),
  map_linguistic_distance = figure_spec("map_linguistic_distance",
   "map_linguistic_distance.png", "Linguistic Distance", kind = if (maps_available)
   "map" else "status", variable = "wavg_ling_degrees"),
  collage_main_maps = figure_spec(
    "collage_main_maps",
    "collage_main_maps.png",
    "Main district-level map inputs",
    kind = "collage",
    inputs = c("map_emi_exposure", "map_consumption_growth", "map_pucca",
     "map_education")
  ),
  collage_iv_region_maps = figure_spec(
    "collage_iv_region_maps",
    "collage_iv_region_maps.png",
    "Instrument and region map inputs",
    kind = "collage",
    inputs = c("map_region", "map_linguistic_distance")
  )
)

if (!maps_available && !identical(cfg$mode, "final")) {
  # Draft-mode diagnostics live outside outputs/figures/main and are explicitly
```

```
    # labeled as diagnostics by figure_output_dir().
    map_specs <- map_specs[c("map_emi_exposure", "map_consumption_growth")]
  }

  map_input_failures <- character()
  if (!maps_available && identical(cfg$mode, "final")) {
    map_input_failures <- c(
      if (length(missing_vars)) paste0("Missing map variables: ", paste(missing_vars,
      ↪  collapse = ", ")),
      paste0("Geometry coverage: ", round(100 * sf_geometry_coverage(district_panel), 1),
      ↪  "%")
    )
  }

  out <- c(out, map_specs)

  if (length(map_input_failures)) {
    attr(out, "legacy_map_input_failures") <- map_input_failures
  }
  attr(out, "district_panel") <- district_panel
  attr(out, "boundaries_2020") <- boundaries_2020
  out
}

#' make ilo trends figure
#'
#' @return A vector of source image paths.
make_ilo_trends_figure <- function(raw_ilo_figures) {
  raw_ilo_figures
}
```

Publication-quality regression and summary tables

Source: `R/output/make_tables.R`

```
is_final_mode <- function(cfg) identical(cfg$mode, "final")

table_status_failures <- function(x, cfg, label) {
  df <- as.data.frame(x)
  ok_status <- c("mapped", "estimated")
  if (!is_final_mode(cfg) || !"status" %in% names(df)) return(character())
  bad <- !is.na(df$status) & !df$status %in% ok_status
  if (!any(bad)) return(character())
  reasons <- character()
  if ("reason" %in% names(df)) reasons <-
  ↪  unique(stats::na.omit(as.character(df$reason[bad])))
  suffix <- if (length(reasons)) paste0(" Reasons: ", paste(reasons, collapse = "; "))
↪  else ""
  paste0("Final table generation requires completed model output for ", label, ".",
  ↪  suffix)
}

legacy_numeric_stats <- function(df, meta, cost_vars = character(), count_vars =
↪  character()) {
  df <- as.data.frame(df)
```

```
  rows <- lapply(seq_len(nrow(meta)), function(i) {
    v <- meta$var[[i]]
    if (!v %in% names(df)) return(NULL)
    x <- suppressWarnings(as.numeric(as.character(df[[v]])))
    ok <- is.finite(x)
    if (!any(ok)) return(NULL)
    stats <- c(
      Min = min(x[ok], na.rm = TRUE),
      `1Q` = unname(stats::quantile(x[ok], .25, na.rm = TRUE)),
      Med = stats::median(x[ok], na.rm = TRUE),
      `3Q` = unname(stats::quantile(x[ok], .75, na.rm = TRUE)),
      Max = max(x[ok], na.rm = TRUE),
      Mean = mean(x[ok], na.rm = TRUE),
      SD = stats::sd(x[ok], na.rm = TRUE)
    )
    z <- data.frame(var = v, label = meta$label[[i]], N = sum(ok), t(stats), check.names
↪  = FALSE, stringsAsFactors = FALSE)
    if ("desc" %in% names(meta)) z$desc <- meta$desc[[i]]
    z
  })
  out <- safe_bind_rows(rows)
  if (!nrow(out)) return(data.frame(var = character(), label = character(), N =
   ↪  integer()))
  for (nm in intersect(c("Min", "1Q", "Med", "3Q", "Max", "Mean", "SD"), names(out))) {
    x <- suppressWarnings(as.numeric(out[[nm]]))
    out[[nm]] <- ifelse(
      out$var %in% count_vars,
      formatC(round(x), format = "f", big.mark = ",", digits = 0),
      ifelse(out$var %in% cost_vars, formatC(x, format = "f", big.mark = ",", digits =
       ↪  2), sprintf("%.2f", x))
    )
  }
  out
}

legacy_categorical_stats <- function(df, meta) {
  df <- as.data.frame(df)
  rows <- lapply(seq_len(nrow(meta)), function(i) {
    v <- meta$var[[i]]
    if (!v %in% names(df)) return(NULL)
    x <- df[[v]]
    x <- x[!is.na(x)]
    if (!length(x)) return(NULL)
    freq <- table(x)
    data.frame(
      var = v,
      label = meta$label[[i]],
      N = length(x),
      Values = paste(names(freq), collapse = ", "),
      Mode = names(freq)[which.max(freq)],
      `% Mode` = round(max(freq) / sum(freq) * 100, 1),
      `Least Freq.` = names(freq)[which.min(freq)],
      `% Least Freq.` = round(min(freq) / sum(freq) * 100, 1),
      check.names = FALSE,
      stringsAsFactors = FALSE
```

```
    )
  })
  out <- safe_bind_rows(rows)
  if (!nrow(out)) data.frame(var = character(), label = character(), N = integer()) else
    ↪ out
}

legacy_summary_group_row <- function(label, columns) {
  row <- as.list(stats::setNames(rep(NA_character_, length(columns)), columns))
  if ("var" %in% columns) row$var <- paste0(".group_", gsub("[^A-Za-z0-9]+", "_", label))
  if ("label" %in% columns) row$label <- label
  as.data.frame(row, check.names = FALSE, stringsAsFactors = FALSE)
}

insert_summary_group <- function(df, label, before_var) {
  if (!nrow(df) || !"var" %in% names(df)) return(df)
  pos <- match(before_var, df$var)
  if (is.na(pos)) return(df)
  rbind(
    if (pos > 1L) df[seq_len(pos - 1L), , drop = FALSE] else df[0L, , drop = FALSE],
    legacy_summary_group_row(label, names(df)),
    df[pos:nrow(df), , drop = FALSE]
  )
}

legacy_significance_stars <- function(p) {
  ifelse(is.finite(p) & p < 0.001, "***",
    ifelse(is.finite(p) & p < 0.01, "**",
      ifelse(is.finite(p) & p < 0.05, "*", "")))
}

format_estimate <- function(estimate, p.value = NA_real_) {
  ifelse(is.finite(estimate), paste0(sprintf("%.3f", estimate),
    ↪ legacy_significance_stars(p.value)), NA_character_)
}

format_se <- function(std.error) {
  ifelse(is.finite(std.error), sprintf("(%.3f)", std.error), NA_character_)
}

regression_display_table <- function(terms, estimates, std_errors, p_values,
 ↪ outcome_label, gof = NULL) {
  rows <- safe_bind_rows(lapply(seq_along(terms), function(i) {
    data.frame(
      Term = c(terms[[i]], ""),
      value = c(format_estimate(estimates[[i]], p_values[[i]]),
        ↪ format_se(std_errors[[i]])),
      stringsAsFactors = FALSE
    )
  }))
  names(rows)[names(rows) == "value"] <- outcome_label
  if (!is.null(gof) && nrow(gof)) {
    names(gof) <- names(rows)
    rows <- rbind(rows, gof)
  }
```

```
  rows
}

first_finite_scalar <- function(value) {
  value <- suppressWarnings(as.numeric(value))
  value <- value[is.finite(value)]
  if (length(value)) value[[1]] else NA_real_
}

format_gof_number <- function(value, digits = 3L, integer = FALSE) {
  value <- first_finite_scalar(value)
  if (!is.finite(value)) return("")
  if (isTRUE(integer)) sprintf("%.0f", value) else sprintf(paste0("%.", digits, "f"),
   ↪  value)
}

model_gof_rows <- function(model, outcome_label) {
  if (is.null(model) || is_model_status_payload(model)) return(data.frame())
  sm <- tryCatch(summary(model), error = function(e) NULL)
  nobs <- tryCatch(stats::nobs(model), error = function(e) NA_real_)
  r2 <- tryCatch(sm$r.squared, error = function(e) NA_real_)
  adj_r2 <- tryCatch(sm$adj.r.squared, error = function(e) NA_real_)
  fstat <- tryCatch(sm$fstatistic[[1]], error = function(e) NA_real_)
  data.frame(
    Term = c("Observations", "R-squared", "Adjusted R-squared", "Model's F-Statistic"),
    value = c(
      format_gof_number(nobs, integer = TRUE),
      format_gof_number(r2),
      format_gof_number(adj_r2),
      format_gof_number(fstat, digits = 2L)
    ),
    check.names = FALSE,
    stringsAsFactors = FALSE
  ) |>
    stats::setNames(c("Term", outcome_label))
}

probit_gof_rows <- function(selection_model, n, outcome_label) {
  rows <- data.frame(
    Term = "Observations",
    value = format_gof_number(n, integer = TRUE),
    stringsAsFactors = FALSE
  )

  # The active participation model is fit with survey::svyglm(), which is
  # design-based rather than maximum-likelihood.  Likelihood-based quantities
  # such as log-likelihood and McFadden's pseudo-R^2 are therefore deliberately
  # omitted for svyglm objects; calling stats::logLik() on svyglm emits the
  # survey package warning that the model was not fitted by maximum likelihood.
  # If this helper is reused for an ordinary glm in diagnostics, the usual ML
  # summary rows are still meaningful and are reported.
  if (inherits(selection_model, "glm") && !inherits(selection_model, "svyglm")) {
    loglik <- tryCatch(stats::logLik(selection_model), error = function(e) NA_real_)
    null_dev <- tryCatch(selection_model$null.deviance, error = function(e) NA_real_)
    dev <- tryCatch(selection_model$deviance, error = function(e) NA_real_)
```

```
    pseudo_r2 <- if (is.finite(first_finite_scalar(null_dev)) &&
 ↪  first_finite_scalar(null_dev) > 0 && is.finite(first_finite_scalar(dev))) {
      1 - first_finite_scalar(dev) / first_finite_scalar(null_dev)
    } else {
      NA_real_
    }
    rows <- rbind(
      rows,
      data.frame(Term = "Log Likelihood", value = format_gof_number(loglik, digits = 2L),
       ↪  stringsAsFactors = FALSE),
      data.frame(Term = "McFadden pseudo-R-squared", value =
       ↪  format_gof_number(pseudo_r2), stringsAsFactors = FALSE)
    )
  }
  stats::setNames(rows, c("Term", outcome_label))
}


numeric_summary <- function(df, variables = NULL) {
  df <- as.data.frame(df)
  if (is.null(variables)) variables <- names(df)
  variables <- intersect(variables, names(df))
  out <- lapply(variables, function(v) {
    x <- suppressWarnings(as.numeric(as.character(df[[v]])))
    x <- x[is.finite(x)]
    if (!length(x)) return(NULL)
    data.frame(variable = v, min = min(x), q1 = unname(stats::quantile(x, 0.25)), median
     ↪  = stats::median(x), q3 = unname(stats::quantile(x, 0.75)), max = max(x), mean =
     ↪  mean(x), sd = stats::sd(x), n = length(x), stringsAsFactors = FALSE)
  })
  out <- safe_bind_rows(out)
  if (!nrow(out)) data.frame(variable = character(), n = integer()) else out
}

categorical_summary <- function(df, variables) {
  df <- as.data.frame(df)
  variables <- intersect(variables, names(df))
  safe_bind_rows(lapply(variables, function(v) {
    x <- df[[v]]
    tab <- sort(table(x, useNA = "ifany"), decreasing = TRUE)
    if (!length(tab)) return(NULL)
    data.frame(variable = v, category = names(tab), n = as.integer(tab), share =
     ↪  round(as.numeric(tab) / sum(tab), 4), stringsAsFactors = FALSE)
  }))
}

table_status_row <- function(model, status = "unavailable", reason = NA_character_) {
  data.frame(
    model = model,
    term = NA_character_,
    estimate = NA_real_,
    std.error = NA_real_,
    statistic = NA_real_,
    p.value = NA_real_,
    status = status,
```

```
    reason = reason,
    stringsAsFactors = FALSE
  )
}

table_first_existing_column <- function(df, candidates) {
  hit <- intersect(candidates, names(df))
  if (length(hit)) hit[[1]] else NA_character_
}

table_column_or_na <- function(df, column) {
  if (length(column) != 1L || is.na(column) || !column %in% names(df)) {
    return(rep(NA_real_, nrow(df)))
  }
  suppressWarnings(as.numeric(df[[column]]))
}

coefficient_frame <- function(model, vcov_matrix = NULL) {
  estimates <- tryCatch(stats::coef(model), error = function(e) NULL)
  if (is.null(estimates) || !length(estimates)) return(data.frame())

  terms <- names(estimates)
  if (is.null(terms) || !length(terms)) terms <- paste0("term_", seq_along(estimates))
  estimates <- suppressWarnings(as.numeric(estimates))

  vc <- vcov_matrix
  if (is.null(vc)) vc <- tryCatch(stats::vcov(model), error = function(e) NULL)
  se <- rep(NA_real_, length(estimates))
  if (!is.null(vc) && length(dim(vc)) == 2L && all(dim(vc) >= length(estimates))) {
    diag_vc <- suppressWarnings(as.numeric(diag(vc)))
    vc_terms <- rownames(vc)
    if (!is.null(vc_terms) && length(vc_terms)) {
      matched <- match(terms, vc_terms)
      ok <- !is.na(matched) & matched <= length(diag_vc)
      se[ok] <- sqrt(pmax(diag_vc[matched[ok]], 0))
    } else {
      se <- sqrt(pmax(diag_vc[seq_along(estimates)], 0))
    }
  }

  statistic <- estimates / se
  statistic[!is.finite(statistic)] <- NA_real_
  df_resid <- tryCatch(stats::df.residual(model), error = function(e) NA_real_)
  p_value <- if (is.finite(df_resid) && df_resid > 0) {
    2 * stats::pt(abs(statistic), df = df_resid, lower.tail = FALSE)
  } else {
    2 * stats::pnorm(abs(statistic), lower.tail = FALSE)
  }
  p_value[!is.finite(p_value)] <- NA_real_

  out <- data.frame(
    Estimate = estimates,
    `Std. Error` = se,
    statistic = statistic,
    `Pr(>|t|)` = p_value,
```

```
    check.names = FALSE,
    stringsAsFactors = FALSE
  )
  rownames(out) <- terms
  out
}

plain_model_coefficients <- function(model) {
  coefficient_frame(model)
}

clustered_model_coefficients <- function(model) {
  tryCatch({
    vc_fun <- clustered_model_vcov(model)
    if (is.null(vc_fun)) return(data.frame())
    vc <- vc_fun(model)
    coefficient_frame(model, vc)
  }, error = function(e) data.frame())
}

filter_table_model <- function(df, candidates) {
  if (!"model" %in% names(df)) return(df)
  out <- df[df$model %in% candidates, , drop = FALSE]
  if (nrow(out)) out else df[0L, , drop = FALSE]
}

is_model_status_payload <- function(x) {
  is.list(x) &&
    !inherits(x, c("lm", "ivreg")) &&
    !is.null(x$status) &&
    length(x$status) == 1L &&
    (is.character(x$status) || is.factor(x$status))
}

model_status_reason <- function(x) {
  reason <- x$reason
  if (is.null(reason) || !length(reason) || all(is.na(reason))) NA_character_ else
   ↪  as.character(reason[[1]])
}

tidy_iv_models <- function(iv_models) {
  if (!is.list(iv_models) || inherits(iv_models, c("lm", "ivreg"))) iv_models <-
   ↪  list(model = iv_models)
  safe_bind_rows(lapply(names(iv_models), function(name) {
    model <- iv_models[[name]]
    if (is_model_status_payload(model)) {
      return(table_status_row(name, as.character(model$status[[1]]),
       ↪  model_status_reason(model)))
    }
    coefs <- clustered_model_coefficients(model)
    if (!nrow(coefs)) coefs <- plain_model_coefficients(model)
    if (!nrow(coefs)) {
      return(table_status_row(name, "unavailable", "Model coefficients are
       ↪  unavailable."))
    }
```

```
    estimate_col <- table_first_existing_column(coefs, c("Estimate", "estimate"))
    se_col <- table_first_existing_column(coefs, c("Std. Error", "std.error",
 ↪  "Std.Error"))
    statistic_col <- table_first_existing_column(coefs, c("t value", "z value", "t", "z",
 ↪  "statistic"))
    p_col <- table_first_existing_column(coefs, c("Pr(>|t|)", "Pr(>|z|)", "p.value", "p",
 ↪  "P>|t|", "P>|z|"))

    if (is.na(estimate_col)) {
      return(table_status_row(name, "unavailable", paste("Model coefficient table lacks
       ↪  an estimate column. Columns:", paste(names(coefs), collapse = ", "))))
    }

    data.frame(
      model = name,
      term = rownames(coefs),
      estimate = table_column_or_na(coefs, estimate_col),
      std.error = table_column_or_na(coefs, se_col),
      statistic = table_column_or_na(coefs, statistic_col),
      p.value = table_column_or_na(coefs, p_col),
      status = "estimated",
      reason = NA_character_,
      stringsAsFactors = FALSE
    )
  }))
}

clustered_model_vcov <- function(model) {
  cluster <- attr(model, "cluster_state")
  if (is.null(cluster)) return(NULL)
  cluster <- as.vector(cluster)
  if (length(cluster) != stats::nobs(model)) return(NULL)
  if (sum(!is.na(cluster)) < 2L || length(unique(cluster[!is.na(cluster)])) <= 1L)
   ↪  return(NULL)
  if (!requireNamespace("sandwich", quietly = TRUE)) return(NULL)
  force(cluster)
  function(x) sandwich::vcovCL(x, cluster = cluster)
}

#' make tables
#'
#' @return A named list of data frames consumed by save_tables().
make_tables <- function(selection_data, ame_results, district_panel, iv_models,
 ↪  first_stage_tests, cfg, selection_model = NULL) {
  cons_iv <- tidy_iv_models(iv_models)
  cons_iv_required <- filter_table_model(cons_iv, c("consumption", "baseline"))
  table_failures <- c(
    table_status_failures(ame_results, cfg, "average marginal effects"),
    table_status_failures(first_stage_tests, cfg, "first-stage regression"),
    table_status_failures(cons_iv_required, cfg, "second-stage IV regression")
  )

  fs_cons <- make_first_stage_table(first_stage_tests, cfg)
  cons_iv_table <- make_second_stage_table(iv_models)
```

```r
  fs_model <- first_stage_table_model(iv_models, district_panel)
  if (!is.null(fs_model$model)) {
    attr(fs_cons, "legacy_model") <- fs_model$model
    attr(fs_cons, "legacy_vcov") <- fs_model$vcov
    attr(fs_cons, "legacy_add_rows") <- fs_model$add_rows
  }
  cons_model <- second_stage_table_model(iv_models)
  if (!is.null(cons_model$model)) {
    attr(cons_iv_table, "legacy_model") <- cons_model$model
    attr(cons_iv_table, "legacy_vcov") <- cons_model$vcov
  }

  out <- list(
    selection_n = data.frame(n = nrow(as.data.frame(selection_data))),
    sum_tbl_probit_quant = make_selection_summary_numeric_table(selection_data),
    sum_tbl_probit_cat = make_selection_summary_categorical_table(selection_data),
    probit_mfx = make_probit_ame_table(ame_results, nrow(as.data.frame(selection_data)),
      ↪ selection_model),
    sum_tbl_iv = make_iv_summary_table(district_panel),
    fs_cons = fs_cons,
    cons_iv = cons_iv_table,
    ame_results = as.data.frame(ame_results),
    first_stage = as.data.frame(first_stage_tests)
  )
  table_failures <- c(table_failures, attr(out$fs_cons, "legacy_table_input_failures"))
  table_failures <- unique(stats::na.omit(table_failures))
  if (length(table_failures)) attr(out, "legacy_table_input_failures") <- table_failures
  out
}

make_selection_summary_numeric_table <- function(selection_data) {
  meta <- data.frame(
    var = c("AGE", "HH_SIZE", "ENROLLMENT_COST", "dmean_num_IS_EDU_FREE",
      ↪ "dmean_num_TUTION_FEE_WAIVED", "dmean_num_RECD_SCHOLARSHIP_STIPEND",
      ↪ "dmean_num_RECD_TXT_BOOKS", "dmean_num_RECD_STATIONERY",
      ↪ "dmean_num_MID_DAY_MEAL_ETC_RECD", "dmean_num_ENROLLMENT_COST"),
    label = c("Age", "Household size", "Enrollment cost (Rs.)", "Educ. free available?
      ↪ (Yes = 1)", "Tuition waived?", "Scholarship/Stipend?", "Textbooks received?",
      ↪ "Stationery received?", "Mid-day meal or more received?", "Avg. district
      ↪ enrollment cost"),
    stringsAsFactors = FALSE
  )
  out <- legacy_numeric_stats(selection_data, meta, cost_vars = "ENROLLMENT_COST")
  insert_summary_group(out, "District-level aggregates:", "dmean_num_IS_EDU_FREE")
}

make_selection_summary_categorical_table <- function(selection_data) {
  meta <- data.frame(
    var = c("SEX", "RELIGION", "SOCIAL_GROUP", "SECTOR",
      ↪ "DIST_FROM_NEAREST_PRIMARY_CLASS", "father_educ"),
    label = c("Sex", "Religion", "Social group", "Urban", "Distance of nearest primary
      ↪ class", "Father's education"),
    stringsAsFactors = FALSE
  )
  legacy_categorical_stats(selection_data, meta)
```

```
}

make_probit_ame_table <- function(ame_results, n = NA_integer_, selection_model = NULL) {
  native_ame <- attr(ame_results, "legacy_marginaleffects", exact = TRUE)
  out <- as.data.frame(ame_results, stringsAsFactors = FALSE)
  if (!"Term" %in% names(out)) out$Term <- out$term
  table <- data.frame(
    Term = out$Term,
    Estimate = format_estimate(
      suppressWarnings(as.numeric(out$estimate)),
      suppressWarnings(as.numeric(out$p.value))
    ),
    `Std. Error` = format_se(suppressWarnings(as.numeric(out$std.error))),
    check.names = FALSE,
    stringsAsFactors = FALSE
  )
  if (!is.null(native_ame)) {
    attr(table, "legacy_marginaleffects") <- native_ame
    attr(table, "legacy_marginaleffects_n") <- n
  }
  if (!is.null(selection_model)) {
    attr(table, "legacy_selection_model") <- selection_model
  }
  table
}

make_iv_summary_table <- function(district_panel) {
  meta <- data.frame(
    var = c(
      "wavg_ling_degrees", "EMIE",
      "npeople_0708", "consumption_0708", "gini_cons_0708", "pct_urban", "avg_hh_size",
      "dependency_ratio", "pct_fem_head", "pct_hindu", "pct_muslim",
      ↪  "pct_other_religion",
      "pct_st", "pct_sc", "pct_obc", "pct_small_land", "pct_medium_land",
      ↪  "pct_large_land",
      "pct_head_illiterate", "pct_head_lit_to_primary", "pct_head_secondary_plus",
      ↪  "pct_pucca",
      "npeople_1718", "consumption_1718", "gini_cons_1718",
      "consumption_pct_change", "gini_change"
    ),
    label = c(
      "Ling. Distance",
      "EMIE", "Population", "Consumption", "Gini of Consumption", "Pct. Urban", "Avg. HH
      ↪  Size",
      "Dependency Ratio × 100", "Pct. Female Head", "Pct. Hindu", "Pct. Muslim", "Pct.
      ↪  Other",
      "Pct. ST", "Pct. SC", "Pct. OBC", "Pct. Small Land-Owner", "Pct. Med. Land-Owner",
      ↪  "Pct. Large Land-Owner",
      "Pct. Head Educ., Illiterate", "Pct. Head Educ., Lit.-Primary", "Pct. Head Educ.,
      ↪  Secondary+", "Pct. Pucca",
      "Population", "Consumption", "Gini of Consumption",
      "Percent change in consumption", "Change in Gini of consumption"
    ),
    desc = c(
      "Average linguistic distance of mother tongue from Hindi",
```

```
      "EMI exposure",
      "Estimated via NSS sample weights",
      "Average household monthly consumption expenditures (Rs.)",
      "Gini coefficient of consumption",
      "Percentage of people in an urban area",
      "Average household size",
      "Ratio of dependents (0-14, 65+) to labor force (15-64), × 100",
      "Percentage of households with a female head",
      "Percentage of Hindus",
      "Percentage of Muslims",
      "Percentage not Hindu/Muslim",
      "Scheduled Tribe",
      "Scheduled Caste",
      "Other Backward Class",
      "Owns 0.005-0.40 hectares",
      "Owns 0.41-3.00 hectares",
      "Owns $\\geq$ 3.01 hectares",
      "Percentage of household heads with educ. level: illiterate",
      "Percentage of heads with educ. level: literate-primary",
      "Percentage of heads with educ. level: above secondary",
      "Percentage in pucca (permanent) homes",
      "Estimated via NSS sample weights",
      "Average household monthly consumption expenditures (Rs.)",
      "Gini coefficient of consumption",
      "Percent change in consumption",
      "Change in the Gini coefficient of consumption"
    ),
    stringsAsFactors = FALSE
  )
  out <- legacy_numeric_stats(district_panel, meta, count_vars = c("npeople_0708",
↪  "npeople_1718"))
  out <- insert_summary_group(out, "From 2001:", "wavg_ling_degrees")
  out <- insert_summary_group(out, "From 2007-08:", "EMIE")
  out <- insert_summary_group(out, "From 2017-18:", "npeople_1718")
  out <- insert_summary_group(out, "From 2007-08 to 2017-18:", "consumption_pct_change")
  out
}
make_first_stage_table <- function(first_stage_tests, cfg = list()) {
  fs <- as.data.frame(first_stage_tests, stringsAsFactors = FALSE)
  required_cols <- c("model", "term", "estimate", "std.error", "p.value", "partial_f",
↪  "partial_p", "status")
  missing_cols <- setdiff(required_cols, names(fs))
  if (length(missing_cols)) {
    msg <- paste("First-stage results are missing columns:", paste(missing_cols, collapse
↪  = ", "))
    if (is_final_mode(cfg)) stop(msg, call. = FALSE)
    return(table_status_row("first_stage", "unavailable", msg))
  }
  if ("model" %in% names(fs) && any(fs$model %in% c("consumption", "baseline"))) fs <-
↪  fs[fs$model %in% c("consumption", "baseline"), , drop = FALSE]
  if (any(grepl("^[0-9]+$", as.character(fs$term)))) {
    msg <- "First-stage coefficient terms are numeric row positions; coefficient names
↪  were lost upstream."
    if (is_final_mode(cfg)) stop(msg, call. = FALSE)
    return(table_status_row("first_stage", "malformed", msg))
```

```
}
status_reasons <- unique(stats::na.omit(as.character(fs$reason[!is.na(fs$status) &
↪  fs$status != "estimated"])))
fs <- fs[fs$status == "estimated" & !is.na(fs$term), , drop = FALSE]
required_terms <- c("wavg_ling_degrees", "(Intercept)")
missing_terms <- setdiff(required_terms, fs$term)
if (length(missing_terms)) {
  if (length(status_reasons)) {
    msg <- paste(status_reasons, collapse = "; ")
    out <- table_status_row("first_stage", "out_of_active_pipeline", msg)
    attr(out, "legacy_table_input_failures") <- paste0("First-stage table lacks
     ↪  required coefficient(s): ", paste(missing_terms, collapse = ", "), ". Reasons:
     ↪  ", msg)
    return(out)
  }
  msg <- paste("First-stage table lacks required coefficient(s):", paste(missing_terms,
↪  collapse = ", "))
  if (is_final_mode(cfg)) stop(msg, call. = FALSE)
  return(table_status_row("first_stage", "unavailable", msg))
}
term_order <- c(
  "wavg_ling_degrees", "consumption_0708", "gini_cons_0708",
  "pct_urban", "avg_hh_size", "dependency_ratio", "pct_fem_head",
  "pct_hindu", "pct_muslim", "pct_st", "pct_sc", "pct_obc",
  "pct_small_land", "pct_medium_land", "pct_large_land",
  "pct_head_lit_to_primary", "pct_head_secondary_plus", "(Intercept)"
)
fs <- fs[order(match(fs$term, term_order), fs$term), , drop = FALSE]
fs <- fs[!is.na(match(fs$term, term_order)), , drop = FALSE]

stat <- fs[fs$term == "wavg_ling_degrees", , drop = FALSE]
f_value <- if (nrow(stat)) first_finite_value(stat, c("partial_f", "legacy_model_f"))
↪  else NA_real_
f_p <- if (nrow(stat)) first_finite_value(stat, c("partial_p", "legacy_model_p")) else
↪  NA_real_
f_row <- data.frame(
  Term = "Instrument's F-Statistic",
  value = paste0(sprintf("%.2f", f_value), legacy_significance_stars(f_p)),
  stringsAsFactors = FALSE
)
nobs_value <- first_finite_value(stat, c("nobs", "n", "N"))
r2_value <- first_finite_value(stat, c("r.squared", "r2"))
adj_r2_value <- first_finite_value(stat, c("adj.r.squared", "adj_r2"))
gof <- safe_bind_rows(list(
  data.frame(Term = "Observations", value = ifelse(is.finite(nobs_value),
   ↪  sprintf("%.0f", nobs_value), ""), stringsAsFactors = FALSE),
  data.frame(Term = "R-squared", value = ifelse(is.finite(r2_value), sprintf("%.3f",
   ↪  r2_value), ""), stringsAsFactors = FALSE),
  data.frame(Term = "Adjusted R-squared", value = ifelse(is.finite(adj_r2_value),
   ↪  sprintf("%.3f", adj_r2_value), ""), stringsAsFactors = FALSE),
  f_row
))

regression_display_table(
  terms = legacy_iv_term_label(fs$term),
```

```
    estimates = suppressWarnings(as.numeric(fs$estimate)),
    std_errors = suppressWarnings(as.numeric(fs$std.error)),
    p_values = suppressWarnings(as.numeric(fs$p.value)),
    outcome_label = "EMI Exposure",
    gof = gof
  )
}

first_finite_value <- function(df, cols) {
  for (col in cols) {
    if (!col %in% names(df)) next
    value <- suppressWarnings(as.numeric(df[[col]][[1]]))
    if (length(value) && is.finite(value)) return(value)
  }
  NA_real_
}

first_stage_table_model <- function(iv_models, district_panel) {
  model <- second_stage_table_model(iv_models)$model
  if (is.null(model) || !inherits(model, "ivreg")) return(list(model = NULL, vcov = NULL,
  ↪  add_rows = NULL))
  terms <- parse_iv_formula_terms(model)
  if (is.null(terms) || !length(terms$regressors) || !length(terms$instruments)) {
    return(list(model = NULL, vcov = NULL, add_rows = NULL))
  }
  endogenous <- setdiff(terms$regressors, terms$instruments)
  if (!length(endogenous)) endogenous <- terms$regressors[[1]]
  if (!length(endogenous) || is.na(endogenous[[1]]) || !nzchar(endogenous[[1]])) {
    return(list(model = NULL, vcov = NULL, add_rows = NULL))
  }
  formula <- stats::as.formula(paste(endogenous[[1]], "~", paste(terms$instruments,
↪  collapse = " + ")))
  data <- add_legacy_iv_aliases(district_panel)
  if (length(setdiff(all.vars(formula), names(as.data.frame(data))))) {
    return(list(model = NULL, vcov = NULL, add_rows = NULL))
  }
  fit <- tryCatch(stats::lm(formula, data = data), error = function(e) NULL)
  if (is.null(fit)) return(list(model = NULL, vcov = NULL, add_rows = NULL))
  vc <- first_stage_vcov(fit, data)
  excluded <- setdiff(terms$instruments, terms$regressors)
  excluded_term <- if (length(excluded)) excluded[[1]] else NA_character_
  wald <- first_stage_wald_test(fit, excluded_term, vc)
  add_rows <- data.frame(
    term = "Instrument's F-Statistic",
    estimate = paste0(formatC(wald$partial_f, digits = 2, format = "f"),
    ↪  legacy_significance_stars(wald$partial_p)),
    stringsAsFactors = FALSE
  )
  list(model = fit, vcov = vc, add_rows = add_rows)
}

second_stage_table_model <- function(iv_models) {
  model <- NULL
  if (is.list(iv_models) && !inherits(iv_models, c("lm", "ivreg"))) {
    hit <- intersect(c("consumption", "baseline"), names(iv_models))
```

```
    if (length(hit)) model <- iv_models[[hit[[1]]]]
  } else {
    model <- iv_models
  }
  if (is.null(model) || is_model_status_payload(model)) return(list(model = NULL, vcov =
   ↪  NULL))
  vc_fun <- clustered_model_vcov(model)
  vc <- if (is.null(vc_fun)) NULL else tryCatch(vc_fun(model), error = function(e) NULL)
  list(model = model, vcov = vc)
}

make_second_stage_table <- function(iv_models) {
  out <- tidy_iv_models(iv_models)
  out <- filter_table_model(out, c("consumption", "baseline"))
  if (!nrow(out)) return(data.frame(Term = character(), `Consumption Growth` =
   ↪  character(), check.names = FALSE))
  model <- NULL
  if (is.list(iv_models) && !inherits(iv_models, c("lm", "ivreg"))) {
    hit <- intersect(c("consumption", "baseline"), names(iv_models))
    if (length(hit)) model <- iv_models[[hit[[1]]]]
  } else {
    model <- iv_models
  }
  regression_display_table(
    terms = legacy_iv_term_label(out$term),
    estimates = suppressWarnings(as.numeric(out$estimate)),
    std_errors = suppressWarnings(as.numeric(out$std.error)),
    p_values = suppressWarnings(as.numeric(out$p.value)),
    outcome_label = "Consumption Growth",
    gof = model_gof_rows(model, "Consumption Growth")
  )
}

legacy_iv_term_label <- function(term) {
  labels <- c(
    "(Intercept)" = "Constant",
    EMIE = "EMIE",
    emie_2007 = "EMIE",
    wavg_ling_degrees = "Linguistic distance",
    consumption_0708 = "Consumption, 2007-08",
    gini_cons_0708 = "Gini consumption, 2007-08",
    pct_urban = "Pct. urban",
    avg_hh_size = "Avg. HH size",
    dependency_ratio = "Dependency ratio",
    pct_fem_head = "Pct. female head",
    pct_hindu = "Pct. Hindu",
    pct_muslim = "Pct. Muslim",
    pct_st = "Pct. ST",
    pct_sc = "Pct. SC",
    pct_obc = "Pct. OBC",
    pct_small_land = "Pct. small land",
    pct_medium_land = "Pct. medium land",
    pct_large_land = "Pct. large land",
    pct_head_lit_to_primary = "Pct. head literate-primary",
    pct_head_secondary_plus = "Pct. head secondary+"
```

```
  )
  out <- unname(labels[term])
  out[is.na(out)] <- term[is.na(out)]
  out
}

make_diagnostic_tables <- function(...) list()
```